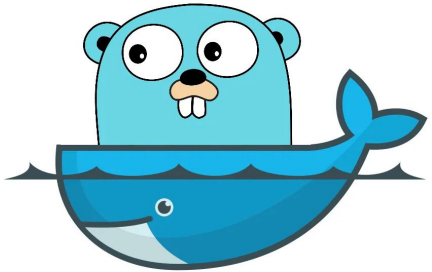


Beyond unit tests: integration testing Go microservices with Docker

Go service + real Postgres, Redis, Kafka in containers

Repo:https://github.com/2beens/go-integration-testing

Today’s Agenda

- Integration testing: the space between **unit** and **E2E**
- What this demo app covers: **Demo Golang App, Kafka, Redis, Postgres, HTTP, idempotency**
- How to **run, debug, break, extend, and ship** integration tests in CI

Pyramid + the testing gap

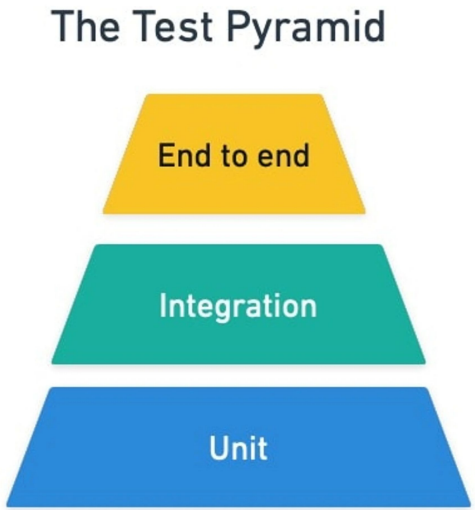
Unit tests - fast, isolated, mock-heavy

Integration tests - real deps, faster feedback, middle layer

E2E - realistic, slow, expensive

- We usually have the two ends - unit & e2e
- The useful middle is often missing

Approach	Speed	Realism
Unit + mocks	high	low
In-process + containers	good	good
Full E2E	low	highest



Benefits (as this demo will show)

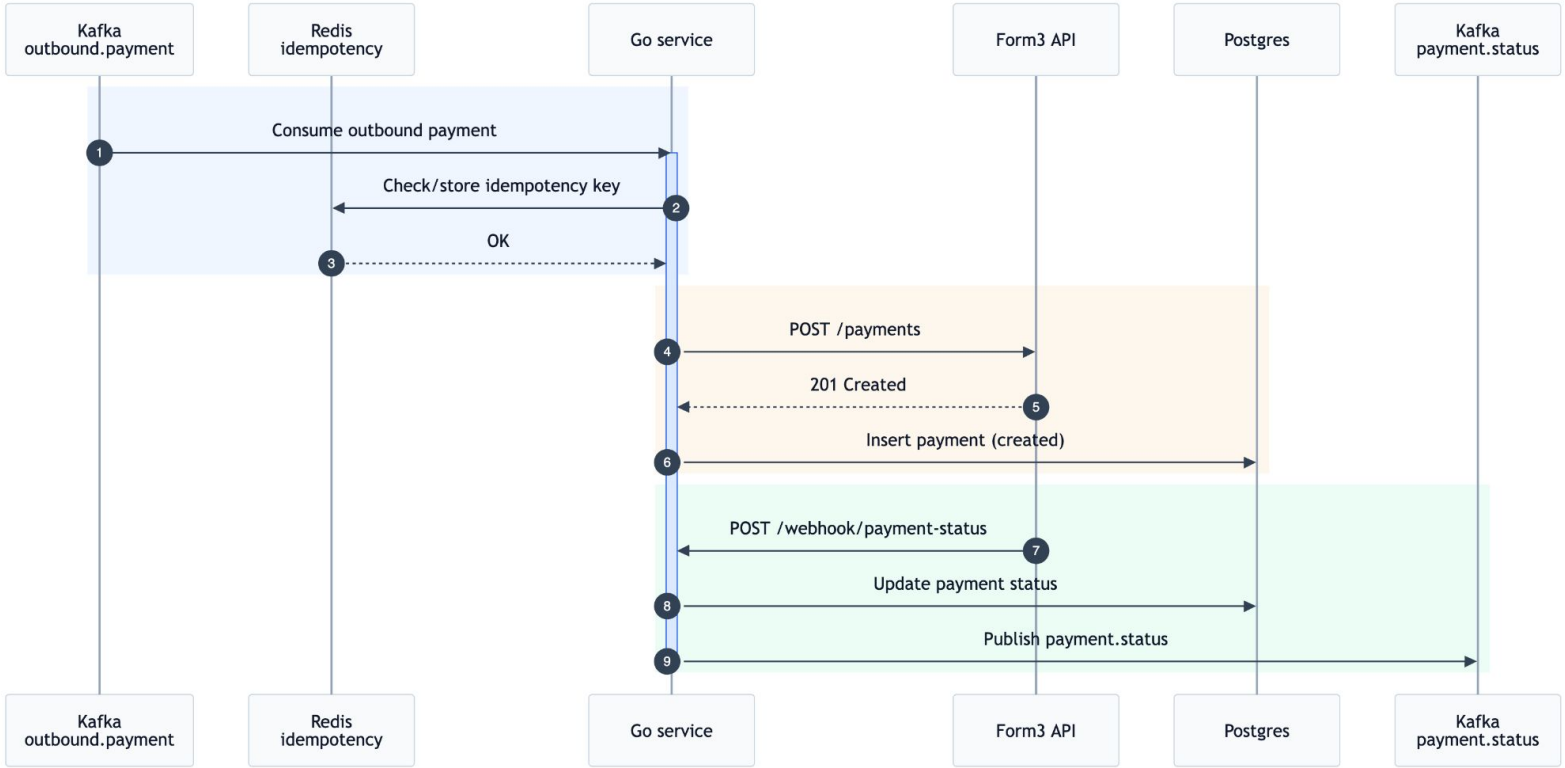
- **Same bootstrap as production** — not a hand-wired test-only router
- **Real protocols** — Postgres, Redis, Kafka, HTTP against a mock Form3
- **Catches wiring mistakes** unit tests often miss (init, config, consumer, migrations)
- **Debuggable** — breakpoints in app + test with one go test launch
- **CI-friendly** — same command as local (RUN_INTEGRATION_TESTS=1)

The payment flow we test

As a demo, we’re using a small Go service for outbound payments.

1. Kafka receives outbound.payment
2. Redis checks and stores idempotency key
3. Service calls Form3 API
4. Service writes payment to Postgres
5. Webhook updates payment status
6. Service emits payment.status

The payment flow we test



Demo — integration package (what matters)

internal/integration/

- **`setup_test.go`** — testcontainers, topics, Form3 mock, **`app.Start`** → `serverURL`
- **`payments_test.go`** — publish to Kafka, assert DB / Redis / mock / HTTP
- **`kafka_test.go`** — Kafka publish / consume helper functions used by the integration tests
- **`form3_mock_test.go`** — mock Form3 HTTP server + request recorder for assertions

Demo — run the tests

- Tests skip when the env var is unset
- Easy to run only when you need them
- Same command shape locally and in CI
- Break a config 🔥 and see what happens - tests fail

```
RUN_INTEGRATION_TESTS=1 go test -v
./internal/integration/...
```

Demo — break bootstrap / init

- Temporarily break something only the **full graph** exercises, e.g.:
  - wrong DSN field in test config, or
  - consumer not started, or
  - migration path / topic setup
- Run `go test ./...`: **unit tests still green**
- Run **integration**: **fails** with a clear error

Point: integration guards **composition**, not just functions.

Demo — extend a test: duplicate idempotency

TestOutboundPayment_DuplicateIdempotency (payments_test.go)

1. Assert Redis **does not** have the key yet
2. **Publish** first outbound.payment → wait for **one** payment + **one** Form3 call
3. Assert Redis **has** the key
4. **Publish duplicates** (same idempotency key)
5. Assert still **one** payment, **same** ID, **no extra** Form3 calls

Demo - Debugging in VS Code / Cursor

- Add RUN_INTEGRATION_TESTS=1 to debug config
- Put breakpoints in test or app code
- Step through startup, DB, Kafka, Redis flow like any other Go test

```
{
  "name": "Debug integration test",
  "type": "go",
  "request": "launch",
  "mode": "test",
  "env": { "RUN_INTEGRATION_TESTS": "1" }
}
```

From local runs to CI

- Add one integration-test job
- GitHub runners already have Docker
- Reuse the same test setup as local development

```
- name: Run integration tests
  run: RUN_INTEGRATION_TESTS=1 \
    go test -timeout=120s ./internal/integration/...
```

When to use this style

Use it for:

- critical flows you really do not want to break
- cases where Postgres, Redis, Kafka, and HTTP all need to work together
- idempotency and making sure the real messages / payloads still work end to end
- making sure events are both produced and consumed correctly
- Integration tests are a middle tier, they do not replace unit tests ⚠️

Pitfalls:

- flakiness from async behavior and timing
- slower test runs and slower feedback in local dev and CI
- isolation and test data cleanup
- dependency on Docker and external images
- maintenance cost if this layer grows too much


THANK YOU
FOR YOUR
ATTENTION